

Sample Questions for CO1 – AT1 – Data Interpretation

1. Employee Salary Analysis

A CSV file (employees.csv) contains employee details.

Task:

1. Read the CSV file.
2. Display employees earning more than ₹50,000.
3. Calculate average salary department-wise.

Code:

```
import pandas as pd
```

```
df = pd.read_csv("../Csv/Employee.csv")
```

```
print("Employees earning more than ₹50,000")
```

```
print(df[df["Salary"] > 50000])
```

```
print("\nAverage Salary Department-wise")
```

```
print(df.groupby("Department")["Salary"].mean())
```

Output:

```
6      107      Arjun      IT      68000
7      108      Meera      HR      58000

Average Salary Department-wise
Department
Finance      51500.000000
HR           51666.666667
IT           67666.666667
Name: Salary, dtype: float64
```

2. Missing Data Handling

Given a CSV file containing student marks with missing values.

Task

1. Identify missing values.
2. Replace missing marks with the column average.
3. Display the cleaned dataset.

Code:

```
import pandas as pd
```

```
df = pd.read_csv("students.csv")
```

```
print("Missing Values")
```

```
print(df.isnull())
```

```
df.fillna(df.mean(numeric_only=True), inplace=True)
```

```
print("\nCleaned Dataset")
```

```
print(df)
```

Output:

```
4    False  False  False  False  False
```

```
Cleaned Dataset
```

	Roll_No	Name	Maths	Science	English
0	1	Rahul	85.00	90.0	88.00
1	2	Priya	78.00	86.0	92.00
2	3	Amit	90.00	85.0	89.75
3	4	Sneha	82.25	89.0	95.00
4	5	Karan	76.00	80.0	84.00

3. Duplicate Record Removal

A customer dataset contains duplicate rows.

Task:

1. Read the dataset.
2. Find duplicate records.
3. Remove duplicates.
4. Display the updated dataset.

Code:

```
import pandas as pd
```

```
df = pd.read_csv("../Csv/customer.csv")
```

```
print("Duplicate Records")
```

```
print(df[df.duplicated()])
```

```
df = df.drop_duplicates()
```

```
print("\nUpdated Dataset")
```

```
print(df)
```

Output:

```
5          101  Rahul    Chennai
Updated Dataset
  Customer_ID  Name    City
0          101  Rahul    Chennai
1          102  Priya  Coimbatore
2          103   Amit   Madurai
4          104  Sneha    Salem
6          105  Karan    Trichy
```

4. Data Transformation

A product dataset stores prices in USD.

Task

1. Convert prices into INR.
2. Add a new column called "Price_INR"

Code:

```
import pandas as pd
```

```
df = pd.read_csv("../Csv/product.csv")
```

```
exchange_rate = 83
```

```
df["Price_INR"] = df["Price_USD"] * exchange_rate
```

```
print(df)
```

Output:

	Product_ID	Product_Name	Price_USD	Price_INR
0	101	Laptop	800	66400
1	102	Mouse	20	1660
2	103	Keyboard	35	2905
3	104	Monitor	250	20750
4	105	Headphones	50	4150

5. Data Aggregation

Given monthly sales data.

Task

1. Calculate total sales.
2. Average sales.
3. Maximum sales.
4. Minimum sales.

Code:

```
import pandas as pd
```

```
df = pd.read_csv("monthly_sales.csv")
```

```
print("Total Sales:", df["Sales"].sum())
```

```
print("Average Sales:", df["Sales"].mean())
```

```
print("Maximum Sales:", df["Sales"].max())
```

```
print("Minimum Sales:", df["Sales"].min())
```

Output:

```
Total Sales: 190000
Average Sales: 31666.666666666668
Maximum Sales: 40000
Minimum Sales: 25000
```

6. Grouping Data

Given employee information.

Task

1. Group employees by department and calculate:
2. Total salary
3. Average salary
4. Employee count

Code:

```
import pandas as pd
```

```
df = pd.read_csv("employees.csv")
```

```
result = df.groupby("Department")["Salary"].agg(  
    Total_Salary="sum",  
    Average_Salary="mean",  
    Employee_Count="count"  
)
```

```
print(result)
```

Output:

	Total_Salary	Average_Salary	Employee_Count
Department			
Finance	103000	51500.000000	2
HR	155000	51666.666667	3
IT	203000	67666.666667	3

7. XML Data Analysis

Given library.xml

Task

Display:

1. Total books
2. Books written after 2020
3. Books costing more than ₹500

Code:

```
import xml.etree.ElementTree as ET
```

```
tree = ET.parse("library.xml")
```

```
root = tree.getroot()
```

```
print("Total Books:", len(root.findall("book")))
```

```
print("\nBooks written after 2020:")
```

```
for book in root.findall("book"):
```

```
    if int(book.find("year").text) > 2020:
```

```
        print(book.find("title").text)
```

```
print("\nBooks costing more than ₹500:")
```

```
for book in root.findall("book"):
```

```
    if float(book.find("price").text) > 500:
```

```
        print(book.find("title").text)
```

Output:

```
Books written after 2020:
Data Science
Machine Learning
Deep Learning

Books costing more than ₹500:
Data Science
Machine Learning
Artificial Intelligence
Deep Learning
```

8. Data Interpretation using Python

Given marks of students.

Task

1. Store marks in a list.
2. Find:
3. Highest mark
4. Lowest mark
5. Average mark
6. Number of students passed.

Code:

```
marks = [78, 65, 90, 45, 35, 88, 55, 72]
```

```
print("Marks:", marks)
```

```
print("Highest Mark:", max(marks))
```

```
print("Lowest Mark:", min(marks))
```

```
print("Average Mark:", sum(marks) / len(marks))
```

```
passed = 0
```

```
for mark in marks:
```

```
    if mark >= 40:
```

```
        passed += 1
```

```
print("Number of Students Passed:", passed)
```

Output:

```
Marks: [78, 65, 90, 45, 35, 88, 55, 72]
Highest Mark: 90
Lowest Mark: 35
Average Mark: 66.0
Number of Students Passed: 7
```


9. Dictionary-Based Analysis

Store employee information using dictionaries.

Task

Display:

1. Highest salary employee
2. Lowest salary employee
3. Average salary

Code:

```
employees = {  
    "Rahul": 45000,  
    "Priya": 60000,  
    "Amit": 55000,  
    "Sneha": 75000,  
    "Karan": 52000  
}  
  
highest = max(employees, key=employees.get)  
lowest = min(employees, key=employees.get)  
average = sum(employees.values()) / len(employees)  
  
print("Highest Salary Employee:", highest, "-", employees[highest])  
print("Lowest Salary Employee:", lowest, "-", employees[lowest])  
print("Average Salary:", average)
```

Output:

```
Highest Salary Employee: Sneha - 75000  
Lowest Salary Employee: Rahul - 45000  
Average Salary: 57400.0
```